

Gabarito - Aula 20 - Programação de Aplicativos

Aqui está o gabarito completo do desafio, dividido entre o arquivo Python (Back-end) e o arquivo HTML (Front-end).

1. O Back-end: app.py

Este arquivo é responsável por criar o servidor Flask, definir a fonte de dados (simulando um banco) e enviar esses dados para o template HTML através da rota principal.

Python

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
# Rota principal (Controller)
```

```
@app.route('/')
```

```
def index():
```

```
    # Fonte de Dados (Simulação de Banco de Dados)
```

```
    musicJ = [
```

```
        {
```

```
            "titulo": "Bohemian Rhapsody",
```

```
            "artista": "Queen",
```

```
            "estilo": "Rock",
```

```
            "fileName": "bohemian_rhapsody.mp3",
```

```
            "duracao": "5:55"
```

```
        },
```

```
        {
```

```
            "titulo": "Smells Like Teen Spirit",
```

```
            "artista": "Nirvana",
```

```
            "estilo": "Grunge",
```

```
            "fileName": "smells_like.mp3",
```

```
            "duracao": "5:01"
```

```
        },
```

```
        {
```

```
            "titulo": "Shape of You",
```

```
            "artista": "Ed Sheeran",
```

```
            "estilo": "Pop",
```

```
            "fileName": "shape_of_you.mp3",
```

```
            "duracao": "3:53"
```

```
        }
```

```
    ]
```

```
    # Injeção de Dependência Visual (Context Passing)
```

```
    # Passamos a variável musicJ para o template 'playlist.html'
```

```
    return render_template('playlist.html', musicJ=musicJ)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

2. O Front-end: templates/playlist.html

Lembrete: Este arquivo **deve** estar dentro de uma pasta chamada `templates` no mesmo diretório do seu arquivo `app.py`, pois o Flask procura os arquivos HTML por padrão lá.

HTML

```
<!DOCTYPE html>
```

```
<html lang="pt-br">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Minha Playlist</title>
```

```
  <style>
```

```
    /* CSS básico para deixar a tabela apresentável */
```

```
    body { font-family: Arial, sans-serif; padding: 20px; background-color: #f9f9f9; }
```

```
    .container { max-width: 800px; margin: auto; background: white; padding: 20px; border-radius: 8px;
```

```
    box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
```

```
    table { border-collapse: collapse; width: 100%; margin-top: 20px; }
```

```
    th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }
```

```
    th { background-color: #007bff; color: white; }
```

```
    tr:nth-child(even) { background-color: #f2f2f2; }
```

```
    .empty-state { color: #666; font-style: italic; padding: 20px; text-align: center; border: 1px dashed #ccc; margin-top: 20px; }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <div class="container">
```

```
    <h1>🎵 Músicas Disponíveis</h1>
```

```
{# Tratamento de Estados Vazios: Verifica se a lista tem itens #}
```

```
{% if musicJ|length > 0 %}
```

```
  <table>
```

```
    {# Cabeçalho Estático #}
```

```
    <thead>
```

```
      <tr>
```

```
        <th>Título</th>
```

```
        <th>Artista</th>
```

```
        <th>Estilo</th>
```

```
        <th>Arquivo</th>
```

```
        <th>Duração</th>
```

```
      </tr>
```

```
    </thead>
```

```
    {# Corpo Dinâmico #}
```

```
    <tbody>
```

```
      {# Loop utilizando o range(length) conforme solicitado no desafio #}
```

```
      {% for i in range(musicJ|length) %}
```

```
        <tr>
```

```
          {# Células de Dados injetando os valores via índice #}
```

```
          <td>{{ musicJ[i]['titulo'] }}</td>
```

```
          <td>{{ musicJ[i]['artista'] }}</td>
```

```
          <td>{{ musicJ[i]['estilo'] }}</td>
```

```

        <td><code>{{ musicJ[i]['fileName'] }}</code></td>
        <td>{{ musicJ[i]['duracao'] }}</td>
    </tr>
{% endfor %}
</tbody>
</table>

```

```
{% else %}
```

```
{# Mensagem exibida caso a lista musicJ esteja vazia #}
```

```
<div class="empty-state">
```

```
    <p>Nenhuma música cadastrada no momento.</p>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</body>
```

```
</html>
```

Como testar o cenário de lista vazia

Para verificar se o tratamento do estado vazio (a condicional `{% if %}`) está funcionando corretamente, basta ir no `app.py` e alterar a lista para que fique assim:

```
Python
```

```
musicJ = []
```

Ao reiniciar o servidor e recarregar a página, a tabela desaparecerá e a mensagem ***"Nenhuma música cadastrada no momento."*** será exibida.